

Objetivo

Crear ConfigMaps mediante métodos imperativos y declarativos para externalizar configuración no sensible. Consumir ConfigMaps como variables de entorno y archivos montados dentro de Pods. Crear y consumir Secrets de tipo Opaque y TLS comprendiendo su codificación y uso seguro. Crear ServiceAccounts, Roles y RoleBindings aplicando el principio de mínimo privilegio. Validar permisos de identidades Kubernetes mediante kubectl auth can-i y comprobar accesos permitidos y denegados.

Requisitos previos

- Haber completado la Práctica 4 y la Práctica complementaria 4.1.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Comprender Pods, Deployments, Services y namespaces.
- Tener conocimientos básicos de manifiestos YAML y codificación base64.
- Disponer de openssl y base64 desde Git Bash.

Duración estimada

75 minutos

Introducción

En esta práctica trabajarás con tres mecanismos fundamentales para configurar y proteger aplicaciones Kubernetes. Utilizarás ConfigMaps para separar configuración no sensible del contenedor, Secrets para almacenar datos confidenciales y RBAC para controlar qué acciones puede realizar una identidad sobre la API. Crearás recursos, los consumirás desde Pods y verificarás los permisos concedidos y denegados mediante kubectl.

Instrucciones

Preparación del directorio y namespace

En esta práctica crearás un namespace dedicado llamado `lab5` y un directorio local donde conservarás los manifiestos relacionados con ConfigMaps, Secrets y RBAC.

Preparar el entorno de trabajo

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor continúa activo antes de utilizar Minikube.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el directorio correspondiente a este laboratorio.

```
mkdir -p /c/LABS/kubernetes/lab5  
cd /c/LABS/kubernetes/lab5
```

 Copiar

- **Paso 3.** Verifica que Minikube se encuentra disponible y que los nodos continúan en estado **Ready**.

```
minikube status  
kubectl get nodes
```

 Copiar

- **Paso 4.** Crea el namespace **lab5** para mantener aislados todos los recursos utilizados durante esta práctica.

```
kubectl create namespace lab5
```

 Copiar

Salida esperada:

```
namespace/lab5 created
```

 Copiar

- **Paso 5.** Establece temporalmente **lab5** como namespace predeterminado del contexto activo para reducir errores durante los comandos posteriores.

```
kubectl config set-context --current --  
namespace=lab5
```

 Copiar

Salida esperada aproximada:

```
Context "<nombre-del-contexto>" modified.
```

 Copiar

- **Paso 6.** Confirma que el contexto actual utiliza `lab5` antes de comenzar con los recursos de configuración.

```
kubectl config view --minify | grep namespace
```

 Copiar

Salida esperada:

```
namespace: lab5
```

 Copiar

Tarea 1. Crear y consumir ConfigMaps

En esta tarea externalizarás configuración no sensible utilizando ConfigMaps. Crearás valores mediante comandos y YAML, y después comprobarás dos formas de consumo desde Pods.

Tarea 1.1. Crear un ConfigMap desde literales

- **Paso 7.** Ejecuta el comando siguiente para crear `app-config` con tres parámetros básicos que simulan la configuración de una aplicación.

```
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=debug \
  --from-literal=APP_PORT=8080 \
  --from-literal=APP_ENV=development
```

 Copiar

Salida esperada:

```
configmap/app-config created
```

 Copiar

- **Paso 8.** Comprueba las claves almacenadas y revisa sus valores para confirmar que Kubernetes registró correctamente el ConfigMap.

```
kubectl describe configmap app-config
```

 Copiar

Salida esperada aproximada:

Name: app-config

Copiar

Namespace: lab5

Data

====

APP_ENV:

development

APP_PORT:

8080

LOG_LEVEL:

debug

Tarea 1.2. Crear un ConfigMap declarativo

- **Paso 9.** Crea el archivo `configmap-nginx.yaml` para almacenar una página HTML y una configuración mínima de NGINX.

```
touch configmap-nginx.yaml
```

Copiar

- **Paso 10.** Abre el archivo en VS Code y agrega el manifiesto siguiente.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: lab5
data:
  nginx.conf: |
    events {}
    http {
      server {
        listen 80;
        location / {
          root /usr/share/nginx/html;
          index index.html;
        }
        location /health {
          return 200 'OK';
          add_header Content-Type
text/plain;
        }
      }
    }
  index.html: |
    <!DOCTYPE html>
    <html lang="es">
    <head>
      <meta charset="UTF-8">
      <title>Kubernetes ConfigMap</title>
    </head>
    <body>
      <h1>Configuración desde ConfigMap</h1>
      <p>Contenido externalizado correctamente.
    </p>
    </body>
    </html>
```

 Copiar

- **Paso 11.** Valida y aplica el manifiesto para crear el segundo ConfigMap.

```
kubectl apply --dry-run=client -f configmap-
nginx.yaml
```

 Copiar

Salida esperada:

```
configmap/nginx-config created (dry run)
```

 Copiar

- **Paso 12.** Aplica el manifiesto validado para crear `nginx-config` dentro del namespace `lab5`.

```
kubectl apply -f configmap-nginx.yaml
```

 Copiar

Salida esperada:

```
configmap/nginx-config created
```

 Copiar

Tarea 1.3. Consumir variables y archivos

- **Paso 13.** Crea el archivo `pods-configmap.yaml`, donde definirás dos Pods que consumirán ConfigMaps mediante mecanismos diferentes.

```
touch pods-configmap.yaml
```

 Copiar

- **Paso 14.** Agrega el contenido siguiente para inyectar `app-config` como variables de entorno y montar `nginx-config` como archivos.

```
apiVersion: v1
kind: Pod
metadata:
  name: config-env-pod
  namespace: lab5
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - |
          echo "LOG_LEVEL=$LOG_LEVEL"
          echo "APP_PORT=$APP_PORT"
          echo "APP_ENV=$APP_ENV"
          sleep 3600
      envFrom:
        - configMapRef:
            name: app-config
      restartPolicy: Never
  ---
apiVersion: v1
kind: Pod
metadata:
  name: config-volume-pod
  namespace: lab5
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
      volumeMounts:
        - name: nginx-config-volume
          mountPath: /etc/nginx/nginx.conf
          subPath: nginx.conf
        - name: nginx-config-volume
          mountPath:
/usr/share/nginx/html/index.html
          subPath: index.html
      volumes:
        - name: nginx-config-volume
```

 Copiar

```
configMap:  
  name: nginx-config
```

- **Paso 15.** Aplica los Pods y espera hasta que ambos recursos estén listos antes de revisar la configuración inyectada.

```
kubectl apply -f pods-configmap.yaml
```

 Copiar

Salida esperada:

```
pod/config-env-pod created  
pod/config-volume-pod created
```

 Copiar

- **Paso 16.** Espera hasta que ambos Pods estén **Ready** para evitar validar configuración mientras los contenedores todavía están iniciando.

```
kubectl wait --for=condition=Ready pod/config-  
env-pod --timeout=60s  
kubectl wait --for=condition=Ready pod/config-  
volume-pod --timeout=90s
```

 Copiar

Salida esperada:

```
pod/config-env-pod condition met  
pod/config-volume-pod condition met
```

 Copiar

- **Paso 17.** Comprueba las variables de entorno cargadas en el primer Pod.

```
kubectl exec config-env-pod -- \  
  sh -c 'echo "LOG_LEVEL=$LOG_LEVEL  
APP_PORT=$APP_PORT APP_ENV=$APP_ENV" '
```

 Copiar

Salida esperada:

```
LOG_LEVEL=debug APP_PORT=8080 APP_ENV=development
```

 Copiar

- **Paso 18.** Comprueba que el segundo Pod utiliza el contenido montado desde `nginx-config`.

```
kubectl exec config-volume-pod -- \
  sh -c 'cat /usr/share/nginx/html/index.html'
```

 Copiar

Salida esperada aproximada:

```
<!DOCTYPE html>
<html lang="es">
...
<h1>Configuración desde ConfigMap</h1>
<p>Contenido externalizado correctamente.</p>
...
</html>
```

 Copiar

IMPORTANTE: En Git Bash se utiliza `sh -c` para que `/usr/share/nginx/html/index.html` sea interpretado dentro del contenedor y no sea convertido a una ruta de Windows.

- **Paso 19.** Verifica el endpoint `/health` directamente desde el contenedor NGINX.

```
kubectl exec config-volume-pod -- \
  sh -c 'wget -qO- http://127.0.0.1/health'
```

 Copiar

Salida esperada:

OK

 Copiar

NOTA: Los valores consumidos como variables de entorno se establecen al iniciar el contenedor. Los volúmenes de ConfigMap tienen un comportamiento diferente, aunque el uso de `subPath` evita actualizaciones automáticas del archivo montado.

Tarea finalizada

Resultado esperado: ConfigMaps creados y consumidos correctamente mediante variables de entorno y archivos montados dentro de Pods.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 1 con IA](#)

Tarea 2. Crear y consumir Secrets

En esta tarea almacenarás información sensible en Secrets y comprobarás cómo Kubernetes entrega automáticamente sus valores decodificados a los contenedores.

Tarea 2.1. Crear un Secret Opaque

- **Paso 20.** Ejecuta el comando siguiente para crear credenciales ficticias de base de datos en un Secret de tipo `Opaque`.

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=labadmin \
  --from-literal=DB_PASSWORD='Lab-S3cret-2026' \
  --from-literal=DB_HOST=postgres-service
```

 Copiar

Salida esperada:

```
secret/db-credentials created
```

 Copiar

- **Paso 21.** Consulta el Secret en YAML para observar que los valores del campo `data` están representados mediante base64.

```
kubectl get secret db-credentials -o yaml
```

 Copiar

Salida esperada aproximada:

```
apiVersion: v1
data:
  DB_HOST: cG9zdGdyZXMvc2VydmljZQ==
  DB_PASSWORD: TGFiLVMzY3JldC0yMDI2
  DB_USER: bGFiYWRTaW4=
kind: Secret
type: Opaque
```

 Copiar

- **Paso 22.** Extrae y decodifica únicamente `DB_USER` para comprobar la relación entre el valor original y la representación almacenada.

```
kubectl get secret db-credentials \
-o jsonpath='{.data.DB_USER}' \
| base64 --decode
echo
```

 Copiar

Salida esperada:

```
labadmin
```

 Copiar

IMPORTANTE: Base64 es una codificación reversible y no proporciona cifrado. Un usuario autorizado para leer Secrets puede recuperar fácilmente sus valores originales.

Tarea 2.2. Crear un Secret TLS

- **Paso 23.** Genera una clave privada y un certificado autofirmado de 30 días para demostrar la estructura requerida por un Secret `kubernetes.io/tls`.

```
MSYS_NO_PATHCONV=1 openssl req -x509 -nodes -
days 30 \
-newkey rsa:2048 \
-keyout tls.key \
-out tls.crt \
-subj "/CN=lab5.local/O=KubernetesLab"
```

 Copiar

IMPORTANTE: `MSYS_NO_PATHCONV=1` evita que Git Bash transforme `/CN=...` en una ruta de Windows como `C:/Program Files/Git/...`.

- **Paso 24.** Comprueba que OpenSSL generó los dos archivos antes de intentar crear el Secret TLS, evitando continuar con archivos inexistentes.

```
ls -lh tls.crt tls.key
```

 Copiar

Salida esperada aproximada:

```
-rw-r--r-- ... tls.crt  
-rw-r--r-- ... tls.key
```

 Copiar

- **Paso 25.** Inspecciona el sujeto y la vigencia del certificado para confirmar que el Common Name corresponde con `lab5.local`.

```
openssl x509 \  
-in tls.crt \  
-noout \  
-subject \  
-dates
```

 Copiar

Salida esperada aproximada:

```
subject=CN=lab5.local, O=KubernetesLab  
notBefore=...  
notAfter=...
```

 Copiar

- **Paso 26.** Crea `tls-certificate` utilizando el certificado y la clave privada que acabas de validar.

```
kubectl create secret tls tls-certificate \  
--cert=tls.crt \  
--key=tls.key
```

 Copiar

Salida esperada:

```
secret/tls-certificate created
```

 Copiar

- **Paso 27.** Consulta únicamente el tipo del Secret para confirmar que Kubernetes lo registró como un recurso TLS y no como un Secret Opaque genérico.

```
kubectl get secret tls-certificate \  
-o jsonpath='{.type}{"\n"}'
```

 Copiar**Salida esperada:**

```
kubernetes.io/tls
```

 Copiar**Tarea 2.3. Consumir el Secret desde un Pod**

- **Paso 28.** Crea `pod-secret.yaml` para consumir el mismo Secret como variables de entorno y como archivo proyectado de solo lectura.

```
touch pod-secret.yaml
```

 Copiar

- **Paso 29.** Agrega el manifiesto siguiente; la proyección `items` crea explícitamente el archivo `db-user`, evitando depender del nombre automático de la clave.

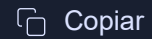
```
apiVersion: v1
kind: Pod
metadata:
  name: secret-demo-pod
  namespace: lab5
spec:
  containers:
    - name: app
      image: busybox:1.36
      command:
        - sh
        - -c
        - |
          echo "DB_USER=$DB_USER"
          echo "DB_HOST=$DB_HOST"

          if [ -n "$DB_PASSWORD" ]; then
            echo "DB_PASSWORD definida
correctamente"
          else
            echo "ERROR: DB_PASSWORD no fue
inyectada"
          fi

          FILE_DB_USER="$(cat /etc/db-secret/db-
user)"

          printf 'Archivo DB_USER=%s\n'
"$FILE_DB_USER"

          sleep 3600
      envFrom:
        - secretRef:
            name: db-credentials
      volumeMounts:
        - name: db-secret-volume
          mountPath: /etc/db-secret
          readOnly: true
      volumes:
        - name: db-secret-volume
          secret:
            secretName: db-credentials
            items:
              - key: DB_USER
```



```
path: db-user
restartPolicy: Never
```

- **Paso 30.** Valida el manifiesto localmente antes de crear el Pod, comprobando sintaxis y estructura sin modificar todavía el clúster.

```
kubectl apply --dry-run=client -f pod-secret.yaml
```

 Copiar

Salida esperada:

```
pod/secret-demo-pod created (dry run)
```

 Copiar

- **Paso 31.** Aplica el manifiesto y espera a que el contenedor quede **Ready** antes de consultar las variables y el archivo proyectado.

```
kubectl apply -f pod-secret.yaml
kubectl wait \
  --for=condition=Ready \
  pod/secret-demo-pod \
  --timeout=60s
```

 Copiar

Salida esperada:

```
pod/secret-demo-pod created
pod/secret-demo-pod condition met
```

 Copiar

- **Paso 32.** Revisa los logs para confirmar que las variables se inyectaron y que el archivo proyectado contiene exactamente el mismo usuario.

```
kubectl logs secret-demo-pod
```

 Copiar

Salida esperada:

```
DB_USER=labadmin
DB_HOST=postgres-service
DB_PASSWORD definida correctamente
Archivo DB_USER=labadmin
```

 Copiar

- **Paso 33.** Valida directamente el archivo montado para demostrar que su contenido proviene del Secret y no únicamente de la variable `DB_USER`.

```
kubectl exec secret-demo-pod -- \
  sh -c 'printf "Contenido del archivo: "; cat
/etc/db-secret/db-user; echo'
```

 Copiar

Salida esperada:

```
Contenido del archivo: labadmin
```

 Copiar

IMPORTANTE: El valor correcto es `labadmin`, porque ese fue el dato creado en `DB_USER`. La práctica no utiliza `labuser`.

Tarea finalizada

Resultado esperado: Secrets Opaque y TLS creados y utilizados correctamente, validando su codificación y consumo seguro desde Pods.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 2 con IA](#)

Tarea 3. Crear una ServiceAccount y un Role de mínimo privilegio

En esta tarea crearás una identidad de aplicación y definirás exactamente qué recursos puede consultar dentro del namespace `lab5`.

Tarea 3.1. Crear la ServiceAccount

- **Paso 34.** Ejecuta el comando siguiente para crear una ServiceAccount denominada `app-reader`, que representará una identidad independiente del usuario administrador.

```
kubectl create serviceaccount app-reader
```

 Copiar

Salida esperada:

```
serviceaccount/app-reader created
```

 Copiar

- **Paso 35.** Comprueba que la nueva identidad aparece junto con la ServiceAccount `default` del namespace.

```
kubectl get serviceaccounts
```

 Copiar

Salida esperada aproximada:

NAME	SECRETS	AGE
app-reader	0	...
default	0	...

 Copiar

NOTA: En versiones actuales de Kubernetes es normal que la columna `SECRETS` muestre `0`; los tokens de ServiceAccount se obtienen mediante TokenRequest y se proyectan en los Pods.

Tarea 3.2. Crear el Role

- **Paso 36.** Crea el archivo `role-reader.yaml` para definir permisos de solo lectura sobre Pods, logs y ConfigMaps.

```
touch role-reader.yaml
```

 Copiar

- **Paso 37.** Agrega las reglas siguientes, excluyendo deliberadamente cualquier permiso sobre Secrets.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-configmap-reader
  namespace: lab5
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]
    verbs: ["get"]
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "watch"]
```

 Copiar

- **Paso 38.** Valida y aplica el Role antes de vincularlo con la ServiceAccount.

```
kubectl apply --dry-run=client -f role-
reader.yaml
```

 Copiar

Salida esperada:

```
role.rbac.authorization.k8s.io/pod-configmap-reader
created (dry run)
```

 Copiar

- **Paso 39.** Aplica el Role después de validar su sintaxis.

```
kubectl apply -f role-reader.yaml
```

 Copiar

Salida esperada:

```
role.rbac.authorization.k8s.io/pod-configmap-reader
created
```

 Copiar

Tarea 3.3. Crear el RoleBinding

- **Paso 40.** Crea `rolebinding-reader.yaml` para asignar las reglas del Role `pod-configmap-reader` a la ServiceAccount `app-reader`.

```
touch rolebinding-reader.yaml
```

 Copiar

- **Paso 41.** Agrega la relación siguiente entre la identidad y el Role.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-reader-binding
  namespace: lab5
subjects:
- kind: ServiceAccount
  name: app-reader
  namespace: lab5
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-configmap-reader
```

 Copiar

- **Paso 42.** Aplica el RoleBinding y revisa su descripción para comprobar que subject y roleRef apuntan a los recursos correctos.

```
kubectl apply -f rolebinding-reader.yaml
```

 Copiar

Salida esperada:

```
rolebinding.rbac.authorization.k8s.io/app-reader-binding created
```

 Copiar

- **Paso 43.** Describe el RoleBinding para verificar que `app-reader` es el subject y que `pod-configmap-reader` es el Role asignado.

```
kubectl describe rolebinding app-reader-binding
```

 Copiar

Salida esperada aproximada:

```
Name:      app-reader-binding
Namespace:  lab5
Role:
  Kind:  Role
  Name:  pod-configmap-reader
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount  app-reader  lab5
```

 Copiar

Concepto clave: Un Role define permisos; un RoleBinding asigna esos permisos a una identidad. Crear únicamente el Role no concede acceso a ningún usuario o ServiceAccount.

Tarea finalizada

Resultado esperado: ServiceAccount y Role configurados con permisos de mínimo privilegio dentro del namespace `lab8`.

Prompt de apoyo para explicar lo realizado[➤ Explicar la Tarea 3 con IA](#)**Tarea 4. Validar permisos con kubectl auth can-i**

En esta tarea comprobarás explícitamente qué operaciones puede realizar `app-reader` y verificarás que los accesos no definidos permanecen denegados.

Tarea 4.1. Validar operaciones permitidas

- **Paso 44.** Ejecuta las consultas siguientes impersonando la ServiceAccount para comprobar que puede leer Pods y ConfigMaps dentro de `lab5`.

```
kubectl auth can-i get pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

```
kubectl auth can-i list pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

```
kubectl auth can-i get configmaps \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

Salida esperada, en el mismo orden de ejecución:

```
yes  
yes  
yes
```

 Copiar

Tarea 4.2. Validar operaciones denegadas

- **Paso 45.** Comprueba que la misma identidad no puede crear Pods, eliminar Pods ni consultar Secrets porque esos permisos nunca fueron incluidos en el Role.

```
kubectl auth can-i create pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

```
kubectl auth can-i delete pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

```
kubectl auth can-i get secrets \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

Salida esperada, en el mismo orden de ejecución:

```
no  
no  
no
```

 Copiar

Tarea 4.3. Comprobar el alcance del namespace

- **Paso 46.** Ejecuta la consulta siguiente para demostrar que el Role creado en `lab5` no otorga automáticamente permisos sobre Pods de `kube-system`.

```
kubectl auth can-i get pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=kube-system
```

 Copiar

Salida esperada:

```
no
```

 Copiar

- **Paso 47.** Utiliza `--list` para obtener una vista consolidada de las acciones que Kubernetes autoriza a la identidad dentro del namespace.

```
kubectl auth can-i --list \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

Salida esperada aproximada:

Resources	Non-Resource URLs	Resource Names
Verbs		
pods	[]	[]
[get list watch]		
pods/log	[]	[]
[get]		
configmaps	[]	[]
[get list watch]		

 Copiar

La tabla puede incluir permisos básicos adicionales de descubrimiento de API, pero no debe conceder lectura de `secrets` ni creación o eliminación de Pods.

NOTA: El formato completo de identidad de una ServiceAccount es `system:serviceaccount:<namespace>:<nombre>`. Este formato es especialmente útil durante auditorías y ejercicios CKA.

Tarea finalizada

Resultado esperado: Permisos permitidos y denegados comprobados correctamente mediante `kubectl auth can-i` utilizando la identidad `app-reader`.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 4 con IA](#)

Tarea 5. Validar RBAC desde un Pod real

En esta tarea asignarás la ServiceAccount al Pod y utilizarás su identidad real para consultar la API del clúster, comprobando tanto operaciones permitidas como denegadas.

Tarea 5.1. Crear el Pod con la ServiceAccount

- **Paso 48.** Crea el archivo `pod-rbac.yaml`, que utilizará una imagen con `kubectl` y ejecutará consultas contra la API con la identidad `app-reader`.

`touch pod-rbac.yaml` Copiar

- **Paso 49.** Agrega el manifiesto siguiente para montar automáticamente las credenciales de la ServiceAccount dentro del Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: rbac-demo-pod
  namespace: lab5
spec:
  serviceAccountName: app-reader
  containers:
    - name: kubectl
      image: bitnami/kubectl:latest
      command:
        - sh
        - -c
        - |
          echo "=== ServiceAccount ==="
          ls
          /var/run/secrets/kubernetes.io/serviceaccount/

          echo ""
          echo "=== Pods permitidos ==="
          kubectl get pods -n lab5

          echo ""
          echo "=== ConfigMaps permitidos ==="
          kubectl get configmaps -n lab5

          echo ""
          echo "=== Secrets denegados ==="
          kubectl get secrets -n lab5 || true

          sleep 3600
```

 Copiar

- **Paso 50.** Aplica el Pod y espera hasta que esté disponible antes de consultar sus logs.

```
kubectl apply -f pod-rbac.yaml
```

 Copiar**Salida esperada:**

```
pod/rbac-demo-pod created
```

 Copiar

- **Paso 51.** Espera hasta que el Pod esté **Ready** antes de leer los resultados generados por kubectl dentro del contenedor.

```
kubectl wait --for=condition=Ready pod/rbac-  
demo-pod --timeout=90s
```

 Copiar**Salida esperada:**

```
pod/rbac-demo-pod condition met
```

 Copiar

- **Paso 52.** Revisa los logs y confirma que la ServiceAccount puede listar Pods y ConfigMaps, pero recibe **Forbidden** al intentar consultar Secrets.

```
kubectl logs rbac-demo-pod
```

 Copiar**Salida esperada aproximada:**

```

=== ServiceAccount ===
ca.crt
namespace
token

=== Pods permitidos ===
NAME                READY   STATUS   ...
...

=== ConfigMaps permitidos ===
NAME                DATA   AGE
app-config          3       ...
nginx-config        2       ...

=== Secrets denegados ===
Error from server (Forbidden): secrets is forbidden:
User "system:serviceaccount:lab5:app-reader" cannot
list resource "secrets" ...

```

 Copiar

La línea `Forbidden` es esperada y demuestra que el Role aplica mínimo privilegio; el `|| true` evita que el contenedor termine por esa comprobación negativa.

Tarea 5.2. Verificar la identidad asignada

- **Paso 53.** Ejecuta la consulta siguiente para comprobar que el Pod utiliza realmente `app-reader` y no la ServiceAccount predeterminada.

```

kubectl get pod rbac-demo-pod \
-o jsonpath='{.spec.serviceAccountName}'{"\n"}'

```

 Copiar

Salida esperada:

```
app-reader
```

 Copiar

Tarea 5.3. Ejecutar la validación final

- **Paso 54.** Copia y ejecuta el bloque siguiente para comprobar los recursos principales y los permisos críticos de la práctica.

```
echo "=== Verificacion final de la Practica 5
=== "

kubectl get configmap app-config >/dev/null 2>&1 \
    && echo "✅ ConfigMap app-config disponible" \
    || echo "❌ ConfigMap app-config no
encontrado"

kubectl get secret db-credentials >/dev/null
2>&1 \
    && echo "✅ Secret db-credentials disponible" \
    || echo "❌ Secret db-credentials no
encontrado"

kubectl get serviceaccount app-reader >/dev/null
2>&1 \
    && echo "✅ ServiceAccount app-reader
disponible" \
    || echo "❌ ServiceAccount no encontrada"

kubectl get role pod-configmap-reader >/dev/null
2>&1 \
    && echo "✅ Role disponible" \
    || echo "❌ Role no encontrado"

kubectl get rolebinding app-reader-binding
>/dev/null 2>&1 \
    && echo "✅ RoleBinding disponible" \
    || echo "❌ RoleBinding no encontrado"

CAN_GET_PODS=$(kubectl auth can-i get pods \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5)

CAN_GET_SECRETS=$(kubectl auth can-i get secrets \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5)

[ "$CAN_GET_PODS" = "yes" ] \
    && echo "✅ app-reader puede leer Pods" \
    || echo "❌ app-reader no puede leer Pods"
```

 Copiar

```
[ "$CAN_GET_SECRETS" = "no" ] \
  && echo "✅ app-reader no puede leer Secrets"
\
  || echo "❌ app-reader tiene acceso inesperado
a Secrets"

SA=$(kubectl get pod rbac-demo-pod \
  -o jsonpath='{.spec.serviceAccountName}')

[ "$SA" = "app-reader" ] \
  && echo "✅ rbac-demo-pod usa app-reader" \
  || echo "❌ ServiceAccount inesperada: $SA"

echo "=== Fin de verificacion ==="
```

Salida esperada aproximada:

```
=== Verificacion final de la Practica 5 ===
✅ ConfigMap app-config disponible
✅ Secret db-credentials disponible
✅ ServiceAccount app-reader disponible
✅ Role disponible
✅ RoleBinding disponible
✅ app-reader puede leer Pods
✅ app-reader no puede leer Secrets
✅ rbac-demo-pod usa app-reader
=== Fin de verificacion ===
```

[Copiar](#)

IMPORTANTE: Conserva los recursos RBAC y el namespace `lab5`. La práctica complementaria siguiente reutilizará la ServiceAccount para profundizar en validación de permisos.

Tarea finalizada

Resultado esperado: Pod RBAC ejecutado con la ServiceAccount `app-reader`, confirmando acceso a Pods y ConfigMaps y denegación sobre Secrets.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 5 con IA](#)

🔧 Resolución de problemas

Problema 1. El Pod muestra CreateContainerConfigError

Síntoma: Un Pod que utiliza ConfigMaps o Secrets no inicia y permanece en `CreateContainerConfigError`.

Causa probable: El Pod referencia un ConfigMap, Secret o clave que no existe dentro del mismo namespace.

Solución:

```
kubectl describe pod config-env-pod
kubectl get configmaps
kubectl get secrets
```

 Copiar

Corrige el nombre del recurso o créalo antes de recrear el Pod.

Problema 2. config-volume-pod no inicia correctamente

Síntoma: El Pod de NGINX entra en `CrashLoopBackOff` o no responde en `/health`.

Causa probable: El contenido de `nginx.conf` tiene un error de sintaxis o el archivo se montó en una ruta incorrecta.

Solución:

```
kubectl describe pod config-volume-pod
kubectl logs config-volume-pod
kubectl get configmap nginx-config -o yaml
```

 Copiar

Corrige `configmap-nginx.yaml`, vuelve a aplicarlo y recrea el Pod si utilizaste `subPath`.

Problema 3. Un valor de Secret no coincide con el esperado

Síntoma: El valor recuperado desde `data` no corresponde con la credencial original.

Causa probable: El valor fue codificado con un salto de línea o existe una diferencia entre el dato creado y el utilizado por el Pod.

Solución:

```
kubectl get secret db-credentials \
  -o jsonpath='{.data.DB_USER}' \
  | base64 --decode
echo
```

[Copiar](#)

Para codificaciones manuales futuras utiliza `echo -n` para evitar incorporar un salto de línea.

Problema 4. kubectl auth can-i devuelve no inesperadamente

Síntoma: `app-reader` no puede realizar una acción de lectura que debería estar permitida por el Role.

Causa probable: El RoleBinding referencia una ServiceAccount, namespace o Role diferente del esperado.

Solución:

```
kubectl get serviceaccount app-reader
kubectl get role pod-configmap-reader -o yaml
kubectl get rolebinding app-reader-binding -o yaml
```

[Copiar](#)

Confirma que `subjects.name` sea `app-reader`, que el namespace sea `lab5` y que `roleRef.name` sea `pod-configmap-reader`.

Problema 5. El Pod RBAC recibe Forbidden incluso para Pods

Síntoma: `rbac-demo-pod` recibe `Forbidden` al ejecutar `kubectl get pods -n lab5`.

Causa probable: El Pod utiliza una ServiceAccount incorrecta o el RoleBinding no está asociado con la identidad realmente utilizada.

Solución:

```
kubectl get pod rbac-demo-pod \
-o jsonpath='{.spec.serviceAccountName}'

kubectl auth can-i get pods \
--as=system:serviceaccount:lab5:app-reader \
--namespace=lab5
```

 Copiar

Si la respuesta continúa siendo `no`, revisa el Role y el RoleBinding antes de modificar el Pod.

Problema 6. Git Bash transforma una ruta Linux del contenedor

Síntoma: Un comando `kubectl exec` intenta abrir una ruta similar a `C:/Program Files/Git/...`.

Causa probable: MSYS2 interpreta una ruta que empieza con `/` como una ruta local de Windows antes de entregarla a `kubectl`.

Solución:

Encapsula el comando que usa la ruta dentro de `sh -c`.

```
kubectl exec config-volume-pod -- \
sh -c 'cat /usr/share/nginx/html/index.html'
```

 Copiar

Para argumentos de herramientas locales como `openssl -subj`, utiliza `MSYS_NO_PATHCONV=1` como se indica en la Tarea 2.2.

Problema 7. localhost devuelve Connection refused dentro de config-volume-pod

Síntoma: NGINX aparece `Running`, pero `wget http://localhost/health` devuelve `Connection refused`.

Causa probable: En este entorno la resolución de `localhost` no produce el comportamiento esperado dentro del contenedor.

Solución:

Utiliza la dirección loopback IPv4 explícita empleada por la práctica.

```
kubectl exec config-volume-pod -- \
  sh -c 'wget -qO- http://127.0.0.1/health'
```

[Copiar](#)

La salida correcta debe ser:

```
OK
```

[Copiar](#)

Problema 8. El archivo proyectado desde db-credentials aparece vacío

Síntoma: Las variables `DB_USER` y `DB_HOST` funcionan, pero la validación del archivo no muestra `labadmin`.

Causa probable: El manifiesto anterior dependía del nombre automático de la clave y mostraba el contenido con un `cat` separado, dificultando distinguir una proyección incorrecta.

Solución:

La versión corregida utiliza una proyección explícita:

```
items:
  - key: DB_USER
    path: db-user
```

[Copiar](#)

Comprueba directamente el archivo:

```
kubectl exec secret-demo-pod -- \
  sh -c 'ls -l /etc/db-secret && printf "DB_USER
archivo="; cat /etc/db-secret/db-user; echo'
```

[Copiar](#)

La salida debe incluir:

```
DB_USER archivo=labadmin
```

 Copiar

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica habrás configurado aplicaciones mediante ConfigMaps y Secrets, consumido sus valores desde Pods y construido un modelo RBAC basado en ServiceAccounts, Roles y RoleBindings, validando de forma explícita los permisos permitidos y denegados.

Notas y consideraciones

- Esta práctica utiliza un namespace dedicado llamado lab5 para mantener aislados los recursos de configuración y seguridad.
- Los valores almacenados en Secrets mediante data están codificados en base64; la codificación no equivale a cifrado.
- Los ejemplos de credenciales utilizados son exclusivamente datos de laboratorio y no deben reutilizarse en sistemas reales.
- La administración de ServiceAccounts se profundizará en la práctica complementaria 5.1.
- En Git Bash, las rutas Linux usadas dentro de contenedores se ejecutan mediante `sh -c` para evitar conversión automática de rutas por MSYS2.
- La variable APP_PORT=8080 del ConfigMap es un dato de configuración demostrativo; no cambia automáticamente el puerto 80 utilizado por NGINX.

URL de referencia

- [ConfigMaps](#)
- [Secrets](#)
- [RBAC Authorization](#)
- [Service Accounts](#)